



MVC: Enabling Fully Coherent Multi-Data-Views through the Memory Hierarchy with Processing in Memory

Daichi Fujiki
Keio University
dfujiki@keio.jp

ABSTRACT

Fusing computation and memory through Processing-in-Memory (PIM) provides a radical solution to the memory wall problem by minimizing communication overheads for data-intensive tasks, leading to a revolutionary shift in computer architecture. Although PIM has demonstrated promising results at different layers of the memory hierarchy, few studies have explored integrating compute memories into the memory management system, specifically in relation to coherence protocol. This paper presents MVC, a framework that leverages existing coherence protocols to enable fully coherent views throughout the memory hierarchy. By introducing coherent views, which are user-defined compact representations of conventional data structures, MVC can minimize data movement and harness the reusability of PIM output. The locality-aware MVC views significantly enhance the performance and energy efficiency of various irregular workloads.

CCS CONCEPTS

• **Computer systems organization** → **Architectures**; • **Hardware** → **Memory and dense storage**.

KEYWORDS

Caches, Processing-in-Memory, Cache Coherence Protocol

ACM Reference Format:

Daichi Fujiki. 2023. MVC: Enabling Fully Coherent Multi-Data-Views through the Memory Hierarchy with Processing in Memory. In *56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '23)*, October 28–November 01, 2023, Toronto, ON, Canada. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3613424.3623784>

1 INTRODUCTION

Modern computer systems face growing limitations due to data movement. Computation is already significantly less expensive than transferring data, and the transition towards more heterogeneous and specialized architecture is intensifying the trends. Therefore, computer systems require innovative methods that substantially decrease data movement; otherwise, data movement will increasingly dominate the execution time and energy consumption of a system.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MICRO '23, October 28–November 01, 2023, Toronto, ON, Canada

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0329-4/23/10...\$15.00

<https://doi.org/10.1145/3613424.3623784>

The concept of Processing-in-Memory (PIM) is gaining popularity due to its potential to alleviate the data movement costs and bridge the performance gap between processors and memory [23]. One approach, near-memory computing, involves placing processing elements in close proximity to memory, thereby reducing the data movement costs [3, 6, 17, 18, 33, 41–43, 47, 61, 64]. Another approach, in-memory computing, exploits the physical characteristics of memory for computing, enabling in-situ computation within the memory array [1, 12, 16, 21, 22, 48, 50, 51].

Computing systems today allocate significant die area to the heterogeneous memory system, which can be leveraged for computing by PIM. Nevertheless, data movement costs remain a concern if the volume of data transmitted through the memory hierarchy remains unmitigated. As pointed out by Lockerman *et al.* [37], PIM is effective when programs exhibit minimal data reuse and when computation and data can be spatially distributed. However, when it comes to applications with significant data locality, forgoing cache hierarchies renders PIM considerably less efficient and even introduces additional complexities, such as synchronization and coherence. Prior research suggests that for many applications, traditional cache hierarchies are significantly more effective than PIM [3, 30, 35, 54, 60].

Additionally, coherence has been perceived as an impediment to PIM implementation, and previous investigations have concentrated on minimizing its detrimental impacts. For instance, in light of caches potentially containing unflushed updates (e.g., new database transactions), prior work has proposed several ways to track and communicate transaction delta between the traditional memory hierarchy and PIM to facilitate PIM's access to the most up-to-date snapshots [5, 55].

While there is a rich body of work on PIM in each memory substrate, only a few studies have examined the integration of diverse compute memories into the existing memory hierarchy, particularly concerning coherence protocols. Recent work on multi-layer PIM approaches [20, 37] has tackled the challenges of exploiting the locality and cacheability of input data in PIM. Nonetheless, it appears that the cacheability of output data generated by PIM has not been investigated to the best of our knowledge.

In this study, we showcase that the PIM-generated cache lines can exhibit substantial reusability under a conventional memory hierarchy, and that a PIM-aware cache coherence protocol can facilitate the investigation of reuse while also considerably alleviating the burden of PIM-side coherence management.

The name MVC (Multi-View Cache/Multi-View Coherence) is derived from a technical term *view* in relational databases. A view is a virtual table defined by a query containing database operators such as filtering and projection. PIM can provide various views from the plain and linear memory address space, specializing in

each workload. For example, straightforward adaptations include generating a columnar database from a row-stored database [58]. This also expands to many interesting software optimizations, such as transforming a linked list into an array by subselecting and vectorizing object parameters in a sequence. However, as with database views, they must be dynamically generated to maintain coherence, making it impossible to store and reuse them in a cache, thus a general PIM approach looked less attractive. In contrast, MVC offers an autonomous solution for maintaining coherence in these generated views, making it possible to safely cache and reuse them in the memory hierarchy. A key insight is that the coherence protocol can now help enable such refined use of PIM.

This work addresses several challenges to enable MVC. *First*, we design a programming frontend that adopts a two-phase commit to create a view snapshot by PIM through the memory hierarchy. PIM primarily performs data collection (or gather/scatter) operations combined with simple computation, thereby avoiding data movement costs for unnecessary data within the cache line of the original address space. Views are requested by memory operations to view addresses, which can be defined by programmers. *Second*, we leverage the existing directory-based cache coherence protocol to automatically invalidate views when updates to the original data are requested. Our protocol allows a view cache line to act as a sharer. Importantly, MVC is designed as an extension to a baseline directory-based protocol, facilitating the integration with the current computing systems. We further conduct formal verification to ensure the view containing data from multiple cache lines in the original space can actually work with distributed MESI protocol. Integrating PIM-generated views into coherence protocol addresses PIM's coherence challenges while enabling the efficient reuse of PIM computation. *Third*, we evaluate multiple architectures and design points to examine various MVC implementations. Our PIM design incorporates state-of-the-art multi-layer PIM to enhance locality awareness. The implementation of the coherence protocol can exploit the existing cache structures to minimize the area cost for required extended sharer lists.

In summary, this work offers the following contributions:

- We design MVC that enables fully coherent data views empowered by a multi-layer PIM stack. A coherent view contributes to data movement reduction, locality-aware reuse of PIM computation, simplified coherence management for programmers, and new software optimizations adaptable to a range of use cases. Our programming interface is versatile and compatible with many existing PIM approaches.
- We demonstrate that MVC can leverage existing directory-based coherence protocol. We provide formal verification for MESI with MVC's coherence extension.
- We explore various MVC designs. The area cost of MVC can be minimized by repurposing the existing cache array structure, and MVC does not affect the critical path of normal cache lookups.
- We evaluate MVC for various irregular workloads investigated in prior work and compare it with general multicore and PIM architectures. Our experimental results show that MVC can provide an overall speedup of 3.27× and improve

energy efficiency by 2.66× with only 1.8% cache area overhead.

2 BACKGROUND AND MOTIVATION

2.1 PIM for Irregular Data Manipulation

Applications that access data in irregular and unpredictable patterns, such as those in machine learning, graph processing, and databases, face significant challenges with data movement. Despite the presence of locality in such workloads, standard techniques for reducing data movement are often ineffective [37]. For example, irregular prefetchers can hide data access latency, but they do not decrease overall data movement. Since the computational intensity of such irregular applications is not necessarily high, traditional accelerator designs may not always furnish sufficient performance gains for production. Unpredictable memory access patterns also make it difficult to utilize other memory system optimizations such as local scratchpads and coalescing.

Processing-in-Memory (PIM) is designed to overcome the memory wall and the data movement inefficiencies of the conventional memory hierarchy by performing computation near or inside memory. While PIM has been investigated in various memory substrates, they generally eliminate the deep cache hierarchies and directly communicate with the memory. This design choice is advantageous for some classes of applications with streaming computation patterns, but it fails to exploit the significant locality present in irregular workloads such as tree lookups [37]. Thus, PIM does not always work well with applications that benefit from the cache hierarchies.

In this section, we present two state-of-the-art approaches to PIM for irregular workloads.

2.1.1 Multi-layer PIM for Locality Exploitation. Livia [37] is a pioneering work that addresses the locality-related issue of the PIM approach mentioned above. One of the representative workloads that Livia tries to optimize for is a balanced search tree. As shown in Figure 1, the tree structure can naturally map to the memory hierarchy, allowing the most frequently accessed nodes to be placed closest to the core.

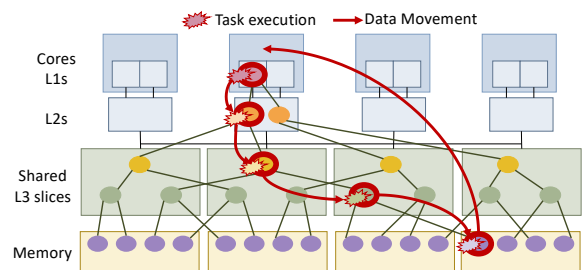


Figure 1: Livia's data mapping and function call cascading [37].

Conventional computers require round trips to the memory system as many times as the tree nodes being traversed, causing harmful moves of less frequently accessed data to the core and evictions of other useful data in upper cache levels. In contrast, Livia performs near-ideal data access by executing and cascading the lookup operation in each layer of the memory hierarchy. The tree lookup

task is divided into a chain of subtasks, where each subtask operates at a single node of the tree and spawns subtasks within the memory hierarchy to visit the child nodes. Livia is particularly useful when the task is composed of a chain of irregular data access and small computations. Computation is performed either by a small in-order core or an FPGA placed near each memory in the hierarchy. Therefore, through data-centric computing across the entire memory hierarchy, Livia can eliminate unnecessary data movement while capturing locality when it is present.

Another approach to utilizing the heterogeneity of memory hierarchy for PIM is explored by MLIMP [20], where different characteristics of memories for in-memory processing are exploited for applications that show various memory footprints and computation patterns.

2.1.2 Data Densification. One of the challenges not addressed by Livia and other locality-aware PIM approaches is the possibly low density of useful data in a cache line. For example, in-memory databases often store a database transaction as a row, which is a tuple containing data for each column in a database schema (e.g. `(user_id=001, name="Alice", age=30, ...)`). This row store is ideal for OLTP; however, analytical workloads (e.g. `avg(age)`) often require only a subset of columns. Such situations are often observed in HTAP (Hybrid Analytical and Transactional Processing) workloads. While conventional and multi-layer PIM can perform analytical workloads extracting and processing the columns of interest, original cache lines containing the row have to be fetched first from the cache (or memory), causing redundant data transfer.

Densifying data at the memory has been another scope for PIM in the prior work [19, 49, 57, 58]. For example, SAM [58] accelerates strided memory access for DRAM based memories, performing the database column densification mentioned above. SAM exploits the under-utilized I/O resources in commodity DRAM chips reserved for designing multiple DRAM modes (e.g. $\times 4$ and $\times 16$) from a common die design. It enables strided gather operations of data in DRAM pages mapped to different subarrays with low-cost modifications to DRAM I/O. Another work [19] densifies CSR format near memory to generate tiles for efficient SpMM in GPUs.

Densification of data brings two benefits: (1) it enables caches to spare more space for important data used by the workloads in flight, and (2) it can reduce data movement of unnecessary data.

2.2 Motivation

PIM data densification can yield nontrivial improvement by incorporating the idea of multi-layer PIM, which allows it to leverage locality of the input. However, a naive implementation of both encounters **input coherence challenges**. Densifying at PIM near the main memory can have stale data and cannot complete data gather operations unless data is guaranteed to remain unchanged or without special coherence support. Densifying near cores does not offer many benefits from densification as long as low-density data is communicated towards the core side. Prior work has partially addressed this issue by managing and sending delta in the cache in order to synchronize the content with that in the lower memory hierarchy where PIM processing takes place [5, 55]. As mentioned, such an approach is suboptimal in view of the concept of the multi-layer PIM.

In addition, we notice **output coherence challenges** have impeded the exploration of output reusability. Generally, densified data is not cacheable. This is because the output has a snapshot of the data at the original addresses, so simply caching and reusing it does not ensure coherence with the original data. This challenge is not addressed by software memoization techniques, either. On the other hand, given a change has been made to only a fraction of the whole dataset, the majority of densified output data can still be reused by another similar task. Such output can also expose locality to the memory system. To the best of our knowledge, most PIM implementations generate single-use output, and none has tried to systematically reuse PIM output in the memory hierarchy, despite the considerable benefits that we demonstrate in our case studies.

These challenges motivate us to develop MVC. To take advantage of the potential locality of PIM-generated data, we overcome the coherence challenges by exploiting the existing coherence protocol, so that we can tie up the addresses of PIM output and the addresses of the original data.

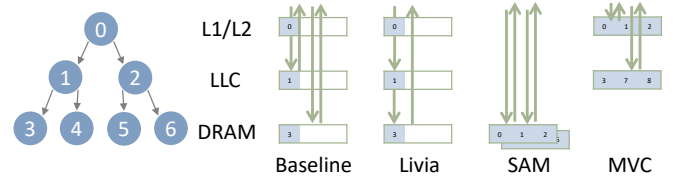


Figure 2: MVC concept compared with prior work.

2.2.1 Motivational Examples. Figure 2 shows a conceptual figure that compares MVC with other approaches in the prior work. Node objects of a binary tree can be randomly allocated in the memory space and can contain various object metadata (e.g., child pointers). Tree algorithms traverse the nodes performing pointer chasing, as shown in the baseline. Livia cascades lookup function through the memory hierarchy, reducing data movement across the hierarchy, while data density and the number of load operations are unaffected. SAM-like architecture can densify data used for tree traversal (e.g., node keys) in subtrees at the main memory. While the number of loads gets smaller, it cannot exploit locality, and the coherence challenges have limited input and output cacheability. MVC can also generate a densified vector view of subtrees by a load request to a view address. Importantly, MVC allows such a view to be cached for future reuses.

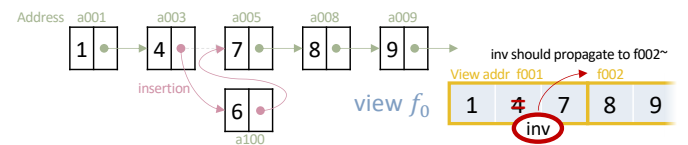


Figure 3: Coherent views for a linked list.

To illustrate the efficacy of coherent views, Figure 3 shows another example of a linked list, which is transformed into a dense vector view with node keys. When a list object is inserted next to node 4, a GetM request is issued to the address for node 4 to modify

the next pointer. MVC allows the coherence protocol to invalidate the view cache line, so that the next request to the same view will be re-generated by PIM using the updated data. While an insertion can also modify the successive views, our optimizations can help support the propagation of invalidation while minimizing its range (Section 4.2).

3 MVC

In this section, we present the MVC execution model. We present the overview of the workflow and the execution model with programming examples. Following that, we discuss MVC's coherence protocol and architectural changes needed to support MVC.

3.1 Overview

The frontend execution model of MVC is designed for a multi-layer PIM system and bears resemblance to the Memory Service in Livia [37]. The code snippet executed in MVC is called MVC service. MVC service is designed to transform data in the original address space into views, performing a simple computation in a PIM module, called MVC PE, in several layers of the memory hierarchy. MVC service also calls MVC API to register the address mapping of view addresses for coherence.

MVC service executes in a cache coherent address space, but MVC service does not restrict the MVC PE implementation to a specific design. MVC PE can be an in-order core or FPGA near memory, or an FSM for in-memory computing.

An MVC service is cross-compiled at the compile time to generate executables for each MVC PE. A view address space segment is allocated in the virtual address space of the program, and the programmer can determine the size and start address.

At the execution time, MVC service binaries are loaded into MVC PE by system firmware, and the view address base is set by the operating system. An MVC service is triggered by a load or store request to the view address. Required arguments are passed to MVC PE using the load/store instruction, while future work can explore other forms of function calls, e.g. x86-64 variadic function call ABI and custom ISA extensions. MVC can also cascade PIM functions as performed in Livia [37]. The result view is returned as a response to the load operation. A store to the memory address included in a view will automatically invalidate cache lines of the dependent views, since MVC allows the view addresses to be registered as the sharers.

3.2 Programming Model and MVC API

Figure 4 shows an example MVC service code snippet for a linked list. This function is called within PIM when the view address state is found to be invalid. This sample program assumes a PIM system with function cascading [37]: if data collection of a view fails, the request propagates to PIM at the lower level of the memory hierarchy.

The `get_ptr` function yields a pointer to the currently traversed list item. Depending on whether the `viewAddr` points to the first view, it returns either the `LIST_HEAD` pointer or the pointer to the prior list item referenced by the preceding view fetch.

`__mvc_transaction_setup` initiates an MVC transaction, altering the coherence state of the view address to the `MVC_SETUP` state,

```
void list_gather (viewTy* viewAddr) {
    listObjTy* ptr = get_ptr(viewAddr-1, LIST_HEAD);
    __mvc_transaction_setup(viewAddr);
    for (int n=0; n < NUM_VIEW_ITEMS; ++n) {
        __mvc_reg_sharer(viewAddr, ptr);
        STATGING_REG[n] = {ptr->id, ptr};
        if (ptr) ptr = ptr->next;
    }
    if (!__mvc_load_commit(viewAddr, STATGING_REG)) {
        __mvc_reset(viewAddr);
        __mvc_invoke(list_gather, viewAddr);
    }
}
```

Figure 4: MVC service snippet for a simple linked list.

as detailed in Section 3.3. Given that MVC services typically interact with multiple cache lines during view creation, the coherence states for the involved cache lines may change during the data collection phase in the for loop. Should the coherence states change, we may have to roll back the changes made to the sharer lists and the controller states, and retry the collection or cascade the request. The `MVC_SETUP` state is used to explicitly indicate that the view address is in the collection phase.

`__mvc_reg_sharer` registers the view address into the sharer list of `ptr` with its coherence state. We will show the details of this implementation in Section 3.3. While invalid cache lines can be registered without state changes, programmers can explicitly insert a load request here to issue `GetS` (force-fetch). Following registration, the `id` and the `ptr` are copied to a temporary register (`STATGING_REG`), and `ptr` advances to the subsequent list item.

`__mvc_load_commit` commits the transaction, by first verifying the view state has not been invalidated. Subsequently, the view address state transitions to a valid state, and a response message for the view request is generated. Should any cache lines involved in the view be in an invalid state, this commit will fail. `__mvc_reset` sets the view state to `MVC_RETIRE` to initiate roll backs of the changes made, and `__mvc_invoke` kicks off a function cascading. The view transaction management is similar to the *two-phase commit (2PC)* of a distributed database.

We also provide `__mvc_check_coherence_state` API to provide information about the current coherence state of a particular cache line. This allows programmers to determine when to force-fetch all participating cache lines or to cascade the request to the next level. While prior work makes the force-fetch decision by weighted coin flip [37], the coherence lookup API offers a more informed and flexible approach.

Note that the fact that this MVC service can restart requires us to ensure the forward progress of the program. Livelock can happen when a node constantly tries to update a cache line while other nodes are creating views simultaneously from it, i.e., under a data race. We can avoid such data races (e.g., by using locks) and limit loop counts before cascade to prevent it.

The MVC service can be executed by an in-order core, a small FPGA, or a near-/in-memory ASIC. Prior work has performed the design space exploration of near-memory acceleration on a similar assumption. Since our work is *orthogonal* to PIM implementation

specifics, we do not compare the PIM implementation in detail. While the selected design points for evaluation include Livia [37] and SAM [58], the scope of our contributions extends beyond these designs.

3.3 Cache Coherence Protocol

In this section, we discuss the MVC cache coherence protocol. Prior work on PIM-aware coherence protocol has put its main focus on *input coherence* management [5, 55]. Since many PIM architectures are designed for the main memory, recent changes made by processors may not be written back to the PIM-enabled memory. LazyPIM [5], for example, performs conflict detection of dataset read or written by PIM. By communicating the read and write dataset to the host after PIM execution, CPU can check if any conflict exists. PIM has to log the original data before computation, and if conflicted, it rolls back the writes and redoes the computation with updated data. Another work [55] bookkeeps updates that affect PIM computation and communicates only delta before initiating PIM.

MVC focuses more on the *output coherence* of the PIM. As discussed in Section 2.2, while data densification is one of the attractive use of PIM, especially when sharing work with host processors, the potential for reuse has not been realized. MVC enables reuse through cache coherence extension.

Our design goals include: (1) building upon existing cache coherence protocols without modifying the underlying protocol to prevent re-verification costs and side effects for caches operating in the original mode, and (2) ensuring minimal additions to the existing cache. We also develop a model to perform formal verification for our extension.

3.3.1 Extended Sharer List. The key idea for MVC extension is to enable a view cache line to be a sharer of a raw (original) cache line. In general, a sharer list is managed by a home node under a directory-based coherence protocol. It is used, for example, when a cache line is shared by multiple child nodes (i.e., a processor or memory node at the processor's side) and an invalidation is requested (e.g., upon receiving GetM); the sharer list is referenced to address invalidation requests to the child nodes. While it is called a list, it is typically implemented as a bit vector stored within a tag array alongside the home state of the cache line.

Adding a view as a sharer introduces an additional (virtual) child node of the raw cache line, allowing the view and its sharer to receive coherence messages. The key challenge lies in managing the sharer list for each view. Unlike physical child nodes of which count is known apriori, an arbitrary number of views can share data in a raw cache line. MVC thus manages the sharer list in an extended sharer list (ESL).

ESL stores a raw line address and its registered views. One possible implementation is based on a normal set-associative cache as depicted in Figure 6. Its set is indexed for a raw address, and the view address is stored in the data array. A part of the raw address is used as a tag. While a typical set-associative cache chooses zero (miss) or one (hit) way from a set on a look-up, ESL can hit several ways if the raw cache line is shared by many views. Visiting each way that hits is enabled by way selection bits (ESL sel). Note that this implementation does not add logic to the critical set index paths.

ESL needs to process sharer list operations to complete coherence requests. For example, to remove a sharer, ESL has to find its raw address entry containing the view address (normal lookup) and remove it. Some requests also need to figure out raw addresses from a view address (reverse lookup). Although one might assume a reverse lookup would require a CAM, it is rather straightforward because (1) raw addresses are often embedded in a view (e.g., ptr in Figure 4) to access raw data, or (2) MVC service typically knows how to generate raw addresses from a view address.

As illustrated in Figure 5 (a), ESL enables a view to be regarded as a sharer. While a sharer is essentially a child node (red arrow), MVC allows a view to occupy a cache entry and be shared by other child nodes (dotted arrows), as depicted. Using the existing mechanism, a view can have its own state and a sharer list as a normal cache line.

Notice that the structure of an ESL is essentially a cache; thus, we can *re-purpose* the existing cache structure (Figure 6). ESL access is further optimized by adding a bit to the original sharer list, indicating ESL has sharers. By doing so, ESL will not be accessed if no view shares the cache line, incurring negligible overhead.

Although not an optimal situation, MVC can introduce an unbounded set of sharers. Our design adopts a *limited directory* approach [10] and restricts the number of sharers. By allowing only a limited number of simultaneously cached copies of any cache lines, limited directory protocols offer scalability for cache directories in large-scale multicore architectures [2, 10]. Upon saturation of the sharer entries, it evicts and reassigns an old entry to satisfy new requests. However, when unbounded sharers are absolutely necessary, this approach can induce constant *thrashing*. LimitLESS [11] emulates a full-map directory in such limited directory hardware implementation by interrupting the processor for software emulation of a full-map directory. While our case studies did not end up requiring an overwhelming number of sharers, these software-aided sharer management approaches can be applied to MVC to fully and flexibly allocate sharers.

3.3.2 MVC States. In the previous section, we discuss a view can be regarded as a child node. Hereafter, we call this child node a view node. Focusing on a single raw address, their relationship can be observed in Figure 5 (b). Figure 5 (c) further illustrates a case with multiple raw addresses in a view.

Since MVC interacts with multiple raw addresses when handling a view, a view cache line necessitates several new transition states. We introduce MVC_SETUP and MVC_RETIRE states to the child node states. MVC_SETUP state signifies that the view address is currently in the data collection phase, undergoing processing by MVC PE. When a view is in MVC_SETUP the cache line state looks identical to the invalid state from other child nodes. Upon committing the MVC transaction, the view states transition to one of the stable states based on the coherence states of the involved raw cache lines. For example, if the baseline protocol is MESI, a view cache line is set to M iff all raw lines are M, E if all are M/E, and S if all are M/E/S; otherwise it is set to I.

The MVC_RETIRE state is set when a rollback process or an invalidation is initiated. On the state change, PutS or PutM is issued for each valid raw cache line. When all lines transition to the invalid (I) state, the view state is altered to I.

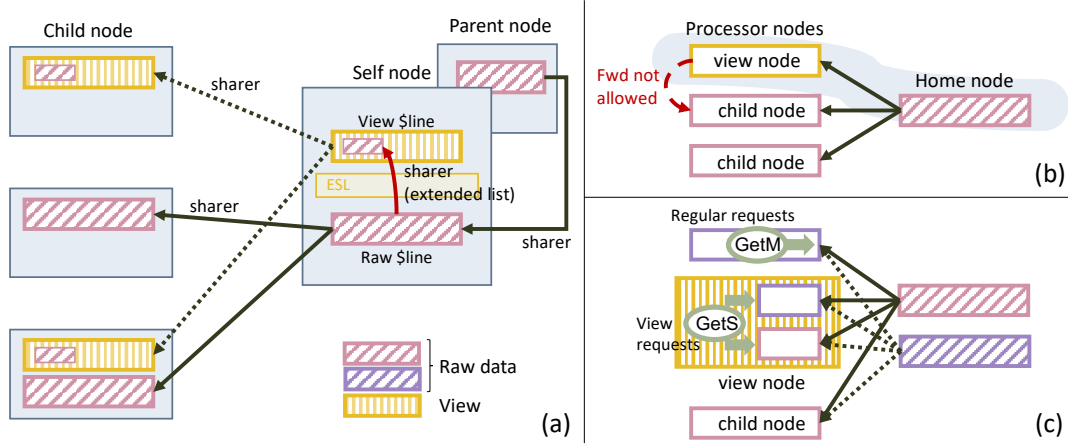


Figure 5: MVC coherence protocol. (a) View registered as a sharer (b) Protocol viewed from a single home node (c) Verification model using multi-tenant view node.

Any coherence request received by the view is handled by itself or propagated to the raw addresses. One exception pertains to forward requests. A forward request is generated to directly share data between child nodes. In the MESI protocol, for example, if a child node *A* has a line in M or E state and another child node *B* issues a GetS, *A* receives FwdGetS from the home node and forwards its data to *B*. However, a view cannot complete the forward request by itself because it only contains a portion of the entire cache line. Fortunately, since the view node is physically situated in the same node as the home node as illustrated in Figure 5, any forward requests addressed to a view can be processed by peeking at the original data in the home node.

3.3.3 Formal Verification. We model the distributed MESI with our coherence protocol extension using Murphi [14] to conduct formal verification. While a model for a view with a single home node can be modeled identically to the model used to verify original MESI as in Figure 5 (b), it cannot examine the state transitions involving multiple raw addresses. Note that MESI can be formally verified using this simplistic model with a single home node and several child nodes, each with one data bit.

We develop a model minimized to check the protocol for a multi-tenant view, shown in Figure 5 (c). It has two home nodes, two child nodes, and a view node with two data slots. The child nodes and the slots in the view node can own data from any home. Any kind of request can be made to any child and view nodes, and the network delivers messages in any possible order. While this model still looks small, adding each extra node and link exponentially increases the possible states, resulting in days of CPU time and hundreds of GBs of memory space for formal verification. To speed up the verification process, we use Eddy Murphi [39], a parallel and distributed version of Murphi, with bit-compaction.

It is worth noting that our coherence extension is built on top of baseline MESI; thus, an actual implementation would not entail significant design and validation costs. To be more conservative, view cache requiring the additional states can also be implemented in a separate cache region to minimize the implementation cost to the existing cache.

3.3.4 Input Coherence. Our MVC extension addresses the coherence challenges of views, i.e. the output of PIM. The PIM operation can be executed outside of the coherent caches, such as near-SSD or near-DRAM. In such a case, it is crucial to guarantee the coherence and consistency of the input, as problematized in the prior PIM coherence work [5, 55]. Fortunately, the multi-layer PIM approach can be extended to provide a lightweight solution to the input coherence challenge. Since many coherence protocols, including MESI, can support self-downgrade, PIM can flush the updated cache lines to the lower memory hierarchy along with a PIM function cascading (with invalidation, if necessary), ensuring any changes become observable to the lower memories. This approach transfers minimum data to perform PIM computation, leading to an efficient implementation of prior work’s approach [55] in hardware.

3.4 MVC Architecture Support

This section presents the MVC architecture, which integrates MVC services and our coherence extension, and discusses its design and implementation.

3.4.1 View Addressing Mode and Contexting. Since view requests go through different control paths, their address must be exposed to the architecture. Some of the challenges that arise include virtual-to-physical address translation and multithreading. View addresses from different thread contexts must be distinguished from each other. Also, since the view address segment is set up in a virtual address space, OS should not allocate a normal physical page for the view segment and should notify the hardware to bypass TLB.

Our approach is similar to VLS [13], which proposes an architecture enabling scratchpad memory in cache. VLS distinguishes the address for scratchpad memory and normal memory by registering the physical address base (pbase) for a scratchpad segment. OS assigns pbase and writes it to a hardware register near TLB during each context switch. When accessing the scratchpad, OS asserts a flag at the MSB of the virtual page address, and hardware sets the pbase as the physical page address, bypassing translation at TLB.

MVC also uses pbase to distinguish view requests from different threads, as shown in Figure 6. The pbase is used as a physical address of views within a given thread context. OS first allocates a view

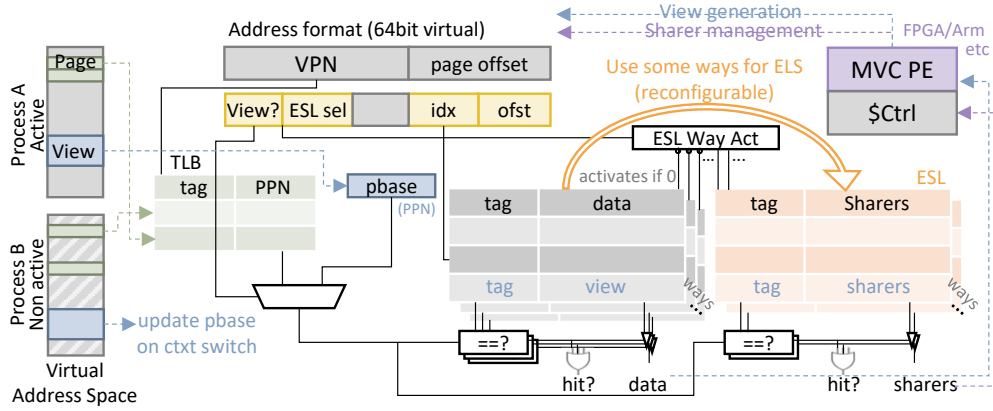


Figure 6: MVC architecture.

address segment in the physical address space upon receiving vmap (view map) system call from a program. During a context switch, OS writes pbase in the hardware register. It enables a thread to issue a view request by asserting a flag in the virtual page address, which allows the MVC PE to determine whether a view is requested.

Virtual Memory: Address translation in MVC PE is needed when dereferencing virtual addresses. Similar to Livia [37], this can be accomplished by sharing L2 TLB (with PEs in cache) and including small TLBs (for PEs in main memory). Memory is assumed to be mapped through large pages, allowing for the use of a small TLB of only a few kilobytes. Such a configuration is frequently employed in data-intensive applications.

Interrupts: Computations in MVC operate independently and within their own context, not adding complexity to precise interrupts on the OoO cores. MVC PE can continue executing during interrupt processing in the host cores because they communicate results through memory. Faults within the MVC PEs are handled by OS at the host, and tasks are returned to the core if necessary.

3.4.2 View Storage Scheme. From an architectural perspective, several MVC implementations can be considered. *View cache* physically stores view content in a cache line and is the most straightforward approach to storing a view in cache. Once a view is generated, it can be treated as a standard cache line and can be copied to upper-level caches without any added complexity. However, *view cache* may have the disadvantage of data redundancy, which occurs when different views reference the same raw address, resulting in multiple copies of data from the raw address in the views. This is primarily due to poorly designed view mapping (i.e., programmers could design mappings to avoid overlaps between views), but it may occur in practice.

Another embodiment, *fragment cache*, does not store view as-is, but scatters cache line fragments using the original cache line addresses rather than the view address. Compacted view cache lines are communicated between memories, but they are scattered/gathered within each memory. The cache fragmentation approach has been investigated for various objectives, such as false sharing mitigation and cache compression [8]. This inevitably increases the hardware complexity since the validity of each fractional cache element must be tracked separately. Moreover, each view read needs

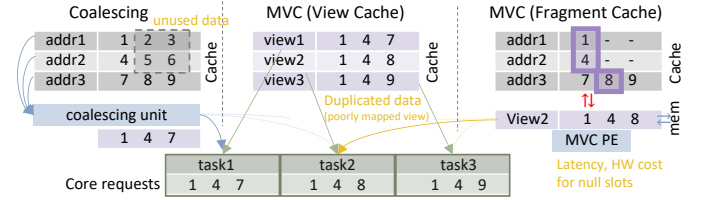


Figure 7: View storage scheme comparisons.

to access multiple cache lines to reconstruct a view, increasing the access latency and energy. However, since duplicated data in views can map to the same physical cache line, the aforementioned redundancy from the overlaps can be avoided.

Fragment cache also requires minor changes to datapaths. If a fragment request is a miss but other fragments for the requesting address exist, the replacement policy must collocate the fragment to the same cache line. Additionally, on a miss, fragments for a view can be mapped to different LLC slices, resulting in multiple requests sent to LLC and the main memory. Since duplicated requests can quickly fill up MSHR and DRAM queues, we introduce *dup request bit* to identify duplications and support multicast of the consolidated fragments to LLC slices for our experiment.

Figure 7 compares two approaches and a traditional coalescing approach, commonly utilized in GPUs to distribute memory content to multiple cores, which can be regarded as view creation at the core side. The coalescing approach can easily reuse data in the cache; however, data movement cost for unused data cannot be reduced. On the other hand, *view cache* can minimize such data movement and memory roundtrips, although duplicated entries may offset the benefits to some extent. *Fragment cache* can achieve both reduced data movement and no redundant copy, but it incurs non-trivial hardware and operational costs in return. Since redundant copies in views can mostly be avoided by defining views without overlaps, MVC employs *view cache* by default.

Fragment cache can also inspire multi-tag cache lines. Since the occupancy per cache row is low, a row can be shared by multiple cache lines with different tags. Multi-tag designs have been developed for cache compression to utilize the saved area [8] and can be applied to *fragment cache* of MVC. However, it incurs non-trivial cost, and further investigation is reserved for future work.

4 APPLICATION CASE STUDY

In this section, we present our application case study illustrating the advantages of MVC. We introduce MVC-aware optimizations for four irregular workloads that are bottlenecked by data movement yet serve as critical components for real-world applications. We will showcase that many such irregular applications can provide useful views within our MVC system, enabling efficient caching and reuse.

4.1 In-Memory Database

In-Memory Databases (IMDBs) are one of the representative applications that require a large number of *strided memory accesses*. IMDBs mainly deal with OLTP (online transactional processing) and OLAP (online analytical processing) workloads. OLTP handles transactions of simple CRUD queries that modify the database, while OLAP focuses on data analysis and retrieval with complex queries.

Storage engines in IMDBs often organize data in a two-dimensional table. A row-oriented layout stores *records* consecutively, which is preferred for OLTP as inserting a record affects minimal address space. A column-oriented layout stores *fields* consecutively, better suited for OLAP. However, neither layout efficiently accommodates a mixed workload of both OLTP and OLAP, known as HTAP, because modern memory is organized in one dimension and HTAP requires strided access to traverse the other dimension. Prior work has proposed PIM designs to accelerate strided memory accesses [49, 57, 58].

MVC provides a view that represents a strided memory access. MVC works ideally because such a view can reduce data movement of unnecessary data, while allowing multiple queries to reuse the view. Frequently accessed views can be cached at higher levels of the hierarchy. In addition, *database views*, which are usually generated dynamically from one or more tables, can naturally map to MVC views, making them cacheable with a “sensitivity list” for invalidation in ESL.

4.2 Linked List and Skiplist

Linked List: Linked lists are often used despite their performance non-ideality. They can be used as standalone data structures or embedded within larger data structures, such as hash maps with separate chaining. They can also be used for low-overhead memory management such as in free lists for an allocator.

A linked list can be viewed as an *array* just by concatenating adjacent keys (and pointers to item objects, optionally), as illustrated in Figure 3. One issue arises when an insertion happens, as it would modify all the successive views, not only the view that inserted node belongs to. We address this issue by making a view of views. Say, f_{001} , f_{002} , $\dots$ are the view addresses of a linked list. We declare a phantom view g_0 and make it dependent on f_{001} , f_{002} , $\dots$. Modifications to raw address a_{002} of f_{001} (Figure 3) will invalidate f_{001} , then g_0 and all views that g_0 is dependent on. However, this approach might not be attractive because a single change would invalidate all views that belong to the linked list. We address this inefficiency by introducing a skiplist.

Skiplist: Skiplist is comparable in complexity to a singly-linked list, but it offers probabilistic complexity guarantees that are similar

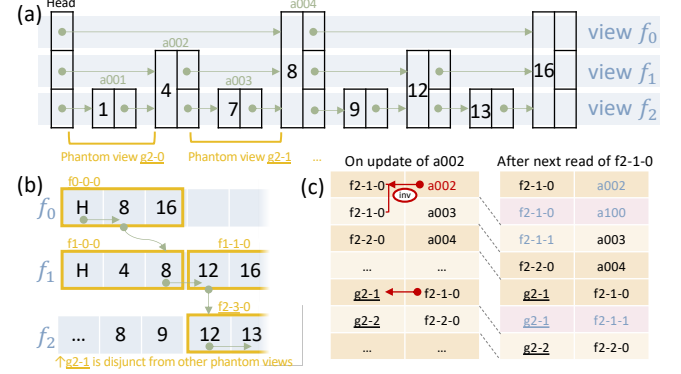


Figure 8: (a) Skiplist overview, (b) view traversal, and (c) MVC optimizations for limiting invalidation chains.

to those of search trees. Due to its simplicity, skiplist has gained popularity as a database storage: Apache Cassandra [9] utilizes it for secondary index memtable implementation, and Redis [46] and WiredTiger [31] utilize them for some storage and in-memory operations.

Figure 8 illustrates the data structure of a skiplist. In a skiplist, nodes of different heights are linked to create hierarchies that allow for skipping ranges of items through express lanes. Each node contains a key, and some nodes have more than one successor, unlike nodes in a linked list. A node with height h is linked from one or more predecessor nodes with a height of up to h . Nodes at the lowest level can be linked from nodes of any height.

A view can provide a mapping to an array of node keys at a certain height. Figure 8 (a) shows view functions mapped to three different heights. For ease of explanation, the address is represented as fh -slice#-line#, where a *slice* is a range of keys under an express lane above them (at height $h - 1$). The view traversal order to access key=13 is illustrated in Figure 8 (b). Pointers to the views in the next level are embedded in each node.

To limit the view invalidation chains observed in linked lists, we map a phantom view to a *slice*. For example, phantom view g_{2-1} depends only on views f_{2-1-*} , so changes to a_{002} , which belongs to f_{2-1-1} , will invalidate views only in g_{2-1} . The other phantom views are disjunct and remain valid. Figure 8 (c) illustrates ESL upon an insertion of key=6 (a_{100}) after key=4. Invalidation is propagated to g_{2-1} , and on the next view access, f_{2-1-0} and f_{2-1-1} are updated and registered for g_{2-1} .

4.3 Binary Search Tree

Binary search trees such as AVL tree remain prevalent data structures, despite facing the challenges for acceleration due to pointer chasing. The node traversal of the original binary search tree exhibits irregular access patterns. Although neighboring nodes may have temporal access locality, the cacheable number of nodes can be lowered by cache pollution by less frequently accessed nodes and metadata of the node object. View provides a condensed key array which significantly improves cache utilization.

To maximize the reusability of a view, a view is composed of node keys in a *triangle subtree* as depicted in Figure 9. A triangle is mapped into an array structure, similar to an array representation

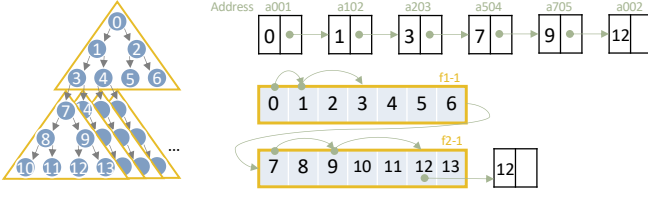


Figure 9: Binary tree and subtree traversal using views.

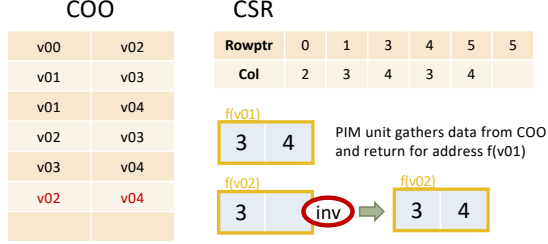


Figure 10: Dynamic graphs and neighbor list views.

of a heap tree. While some keys in views are not visited for one traversal as in Figure 9, we will demonstrate the enhanced reusability benefits more than amortize the cost. Cache pollution can also be minimized by multi-layer PIM.

4.4 Dynamic Graph

Processing dynamic graphs, which involves computing on graphs with continuously changing vertices and edges, has become an essential computational requirement. Dynamic graph algorithms enable real-time optimization solutions and a variety of data-mining applications. However, these algorithms rely heavily on memory and can quickly become memory-bound.

Due to its dynamically changing nature, COO format is suitable for storage, providing simple graph manipulation operations (e.g., insertion). However, it is not suitable for computation, and typically it is sorted and converted to another compute-friendly format, such as CSR, before execution. PIM can provide a view that transparently converts the sparse graph format. A similar PIM approach of converting from storage-friendly to compute-friendly formats has been explored in [19], and MVC can add support for auto-invalidation upon changes in the original format, making it suitable for dynamic graphs. Figure 10 shows a view $f(vtx)$ that lists vertex keys connected to vtx , which auto-reflects an addition of an edge $v02 \rightarrow v04$.

5 METHODOLOGY

Benchmarks: We implemented four irregular workloads presented in the case study. We evaluate these workloads on different data sizes, system sizes, and access patterns. For in-memory databases (IMDBs), we use a representative subset of SQL queries evaluated in the prior work [57, 58], as shown in Table 1. Q1 and Q2 have different selectivity, 20% and 80%, respectively. In addition to queries with selection, projection, and updates, we also include typical but complex queries that select tuples smaller than the field average.

For skiplist, we used memory aligned list object (64B/item). The logarithm base is set to 16. To perform a lookup, we generate a key

Table 1: IMDB benchmark queries.

No.	SQL Statement
Q1	SELECT avg(f9) FROM Ta WHERE f10 > x;
Q2	SELECT avg(f9) FROM Ta WHERE f10 < x;
Q3	SELECT f11 FROM Ta WHERE f9 < (SELECT avg(f9) from Ta);
Q4	SELECT f8, f9, f10 FROM Ta WHERE f9 < (SELECT avg(f9) FROM Ta WHERE f10 > x);
Q5	UPDATE Ta SET f3 = x, f4 = x WHERE f10 = x;

Table 2: System parameters.

Cores	16 cores, x86-64 ISA, 2.4GHz, OOO
Caches	H_3 hash based indexing, MSHR size 16
L1	32KB, 8-way set-associative, split data and instruction
L2	512KB, 8-way set-associative
LLC	32MB (2MB per tile), 16-way set-associative
NoC	mesh, 128-bit flits and links, 2/1-cycle router/link delay
Memory	4 DRAM controllers; DDR4-2400 timing

and scan the skiplist. Keys are chosen randomly, following either a uniform random (ur) or Zipfian (zipf) distribution ($s = 1.11$). Binary search trees (AVL tree) assume the same object setup for tree nodes (except for the node fanout set to 2) and access patterns. The total size of skiplist and binary search tree is set to 128MB and 512MB.

For dynamic graphs, we use real-world dynamic graphs, ubuntu and twitter, from a dynamic graph benchmark suite collection, DyGraph [38], and run BFS on the graphs. For each snapshot, we run 20 BFS queries to obtain 3-hop subgraphs of randomly selected node keys (ur or zipf), assuming a graph server workload for GNNs [56, 62, 63]. On a snapshot update, we update the graph and COO data structure along with newly added nodes and edges.

Simulation setup: We evaluate MVC using a modified ZSIM microarchitecture simulator [45], in which we implement our core, cache/memory system, and coherence controller models. We feed extensive execution traces that include dependency information to our integrated core backend replayer. We use H_3 hash family [44] to index caches to avoid cache thrashing originating from non-uniform view addressing patterns. For MVC, a part (2 ways by default) of LLC is repurposed for ESL except where specified otherwise. The performance model is validated using microbenchmarks. Our system parameters are summarized in Table 2.

We compare the workloads across five systems shown in Table 3.

Table 3: Configurations studied.

System	Description
CPU	A baseline system with a passive cache hierarchy, performing tasks on cores with normal load/store.
PIM	A SAM-like [58] near-memory system that runs MVC service within memory controllers, skipping L2 and LLC.
MLPIM	A Livia-like [37] multi-layer PIM system that runs MVC service through the PE attached to each layer in the memory hierarchy. We use a sampling (gather unrolling) probability of 1/32.
MVC (Frag)	Our proposed design with MVC implemented as a fragment cache.
MVC (View)	Our proposed design with MVC implemented as a view cache.

We report the execution time and dynamic execution energy results, utilizing the energy parameters extracted from [53]. Our focus is on dynamic energy as static power is minimally impacted by MVC. This allows for a clear differentiation between the impact on data movement energy and its overall benefits on performance.

The area and power of PIM and way-based partitioning (ELS) have been shown to be small in the prior work [13, 37, 58], and we use their numbers in our evaluations. We use CACTI 6.5 [40] to model cache and tag arrays modified for MVC, and the cache area overhead turns out to be 1.8%.

6 RESULTS

6.1 Application Performance

In this section, we discuss the performance of our case study applications with MVC. Figure 11 shows the execution time comparison, and Figure 12 shows the dynamic energy comparison. We will discuss the performance for each workload below.

IMDB: In-Memory Databases generally benefit from PIM by extracting and communicating fields of interest to the host processor. Q1 and Q2 do not have data reuse, making PIM outperform other alternatives. MLPIM's sampling scheme unrolls the gather operation to stage data to upper-level memory, but without data reuse, fetching un-gathered data does nothing but add cost for loading unnecessary data, resulting in performance overhead. MVC's overhead primarily originates from coherence-checking operations. Comparing Q1 and Q2, MVC (Frag) outperforms MVC (View) for Q2. In situations with plenty of view requests without reuse, MVC (Frag) can exploit larger bank-level parallelism for fragments, making it slightly more efficient in managing view coherence than MVC (View), which has only two banks for ESL. Q3 and Q4 exhibit data reuse after calculating the field average, making MVC (View) outperform all the other alternatives. This trend prevails in Q4 where more than half of the involved fields can be reused. PIM and MLPIM are less performant because of the un-cacheability of gathered data. Q5 performs gather/scatter operations during updates. While the absence of reuse makes all PIM performance numbers look the same, MVC (Frag) outperforms due to its ability of item-wise invalidation. Dynamic energy comparison shows a trend similar to execution time, while the effect of data movement reduction is more apparent.

Skiplist: Skiplist demonstrates a large reusability of nodes involved in express lanes of higher levels. PIM cannot exploit the reuse opportunity and faces difficulty providing performance benefits. The vector view produced by MVC significantly improves the execution time, yielding a 3.26× speedup. The primary sources of the performance gain are (1) reduced data movement by compact view containing only keys, and (2) high reusability of the view. It is also notable that MLPIM and MVC (Frag) are less efficient. This is likely due to the gather operations of keys not always being self-contained in an LLC slice, necessitating visits to multiple slices. Although such requests are eventually fulfilled in upper-level caches, they need to spawn multiple load requests, resulting in congested access paths and large queuing delays. Furthermore, we use a larger list size than the prior work of MLPIM, which assumes lists fit within LLC. A large linked list also increases the number

of DRAM access due to un-cacheability, causing lower energy efficiency. When the list size is small enough, we observe performance that aligns with the prior MLPIM.

Zipf distribution enhances temporal locality; thus, all systems experience performance improvement in absolute terms. Increased cache effectivity benefits MLPIM and MVC (Frag), yet MVC (View) continues to outperform its counterparts.

AVL tree: AVL tree has a performance trend similar to Skiplist because both share the same probabilistic complexity in big-O notation. PIM experiences about 2× latency due to its inability to exploit locality, consistent with the prior work's findings [37]. While MVC (View) achieves 2.14× speedup over CPU, in comparison to Skiplist, the performance gap between MLPIM and MVC is small. This is because MVC creates a view for triangle subtrees, where not all nodes are visited by a traversal. While triangles offer good reusability for nodes near the top, as untaken paths are likely to be visited soon, they offer no advantages for infrequently accessed nodes near the bottom, resulting in memory access overheads. On the other hand, MLPIM performs node-to-node traversal, while entire node objects are fetched. We compare them in more detail in Section 6.2 using different tree sizes. The weak energy performance is also likely to stem from the same reason. Zipf distribution increases reusability, which helps improve the performance of MVC, as is the case for Skiplist.

Dynamic Graph BSF: MVC (View) demonstrates significant performance improvement for dynamic graph BFS, achieving 4.5× and 8.8× speedup over CPU for ubuntu and twitter graphs, respectively. Interestingly, while twitter is only 1.6× smaller than ubuntu, it exhibits a larger performance improvement. Our analysis reveals that, although the average portion of updated edges per snapshot is comparable for both (3.2% for ubuntu and 3.4% for twitter), ubuntu has much higher connectivity (Avg. degree: 6.7 vs. 2.5, Avg. shortest path length: 3.2 vs. 5.2). This leads to edge coverage of 3-hop subgraphs to be 56% (ubuntu) and 6.4% (twitter), increasing the chance of loading invalidated edge list after snapshot updates for ubuntu. In other words, updates do not propagate much for twitter, increasing the reuse opportunity of cached views. The poor energy efficiency of CPU and traditional PIMs is partly due to increased coherence traffic [37], whereas MVC (View) requires fewer invalidation messages in total thanks to views which can group multiple invalidations.

Overall, MVC (View) achieves 3.27× speedup and 2.66× energy efficiency on average over the CPU baseline. It also achieves a 1.60× speedup compared to the state-of-art MLPIM. As we expected, MVC (Frag) provides performance similar to MLPIM in our case study. In some cases where view address overlap happens non-trivially or fine-grained update is required (e.g., DyBFS ubuntu and IMDB-Q5), it outperforms MLPIM or MVC (View). Nonetheless, MVC (View) has been consistently better in most cases. The performance gains of MVC primarily stem from unlocked reusability of PIM output.

6.2 Input Size Sensitivity

Figure 13 illustrates the impact of varying input sizes on the execution time of AVL and Skiplist. The execution time is normalized to the smallest input and the number of queries is kept the same for all experiments. Owing to the logarithmic scaling of both structures,

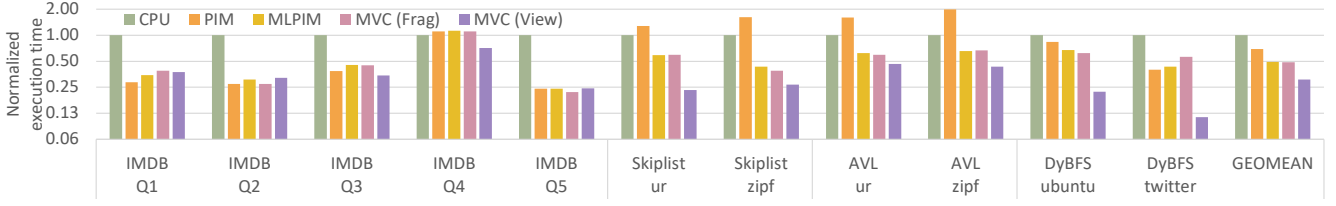


Figure 11: Execution time normalized to our CPU baseline (lower is better).

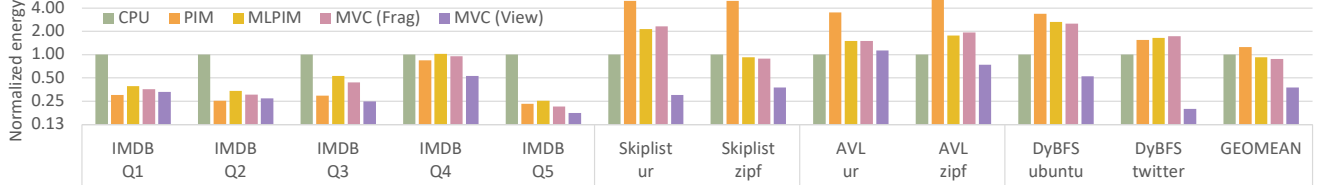


Figure 12: Dynamic energy normalized to our CPU baseline (lower is better).

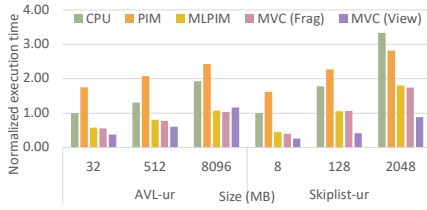


Figure 13: Input size sensitivity.

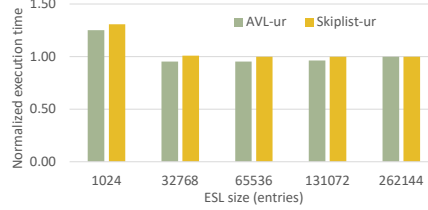


Figure 14: Performance vs. ESL size.

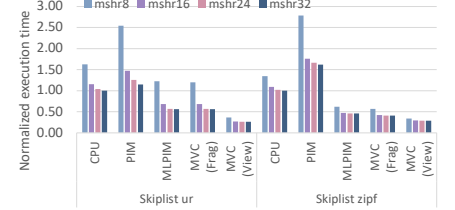


Figure 15: Performance vs. MSHR size.

differences become apparent with each exponential increase in size. As the depth of the tree or skiplist increases, we observe a gradual increase in the execution time. We can see that CPU has the steepest increase rate, and with a large dataset that pollutes the caches to a degree that precludes capturing locality, general PIM outperforms CPU, as shown in the 2GB Skiplist.

For AVL, MVC (View) underperforms compared to MLPIM for the largest dataset. As previously discussed, a view representing a triangle subtree speculatively loads untraversed nodes for future access, incurring performance overhead if the node access frequency is low (e.g., nodes near the bottom). To address this limitation, we propose to combine the node-to-node traversal of MLPIM with MVC: we perform view loads for the top-level nodes until the cache is filled, then perform node-to-node traversal at the PIM in the main memory. This hybrid approach maximizes the reuse of views and minimizes redundant loads. We observe this optimization leads to near-ideal performance scaling, similar to that of Skiplist.

6.3 System Size Sensitivity

ESL size: Figure 14 presents the performance of MVC (View) across various ESL capacities. ESL capacity limits the number of raw addresses whose content is shared by views. 32K entry ESL represents a way of an LLC slice. Execution time is normalized to the design point with the largest ESL. We assume LLC is repurposed for ESL except in the 1KB design. We observe that allocating one or two ways is enough for performance, and it is not very sensitive to ESL

capacity. In fact, allocating more ESL reduces the LLC capacity in return; thus 32K to 64K entry ESL yields the best performance. It is worth noting that, although sensitivity to ESL size turns out to be small, the ESL/LLC ratio can be flexibly adjusted through the way-based partitioning scheme (Section 3.4).

MSHR size: Gather operation at PIM devices slows down the load request, and we observe miss status handling registers (MSHR) are likely to fill up quickly, stalling new memory requests from the upstream (CPU's side). This effect is more severe for MVC (Frag) because it can multicast the requests if items in a view map to multiple LLC slices.

Figure 15 illustrates MSHR size sensitivity to the performance of skiplist-ur and skiplist-zipf. As shown in the figure, an increased MSHR size generally helps improve the performance, especially for PIM, which requires a long round-trip latency to the memory for each access.

Locality-aware PIM approaches, MLPIM and MVC, saturate the performance after sufficient MSHR allocation. We observe that as the cache hit rate increases, backpressure to MSHR decreases. Consequently, even with the extended memory fetch latency (common for all PIM), MVC (View) achieves the highest cache hit rate, and the typical MSHR size is sufficient to obtain the maximum performance. This observation is further supported by skiplist-zipf, which inherently exhibits a larger cache hit rate due to power-law distributed access patterns. Beyond the saturation point, the absolute number of memory requests contributes to performance differences.

6.4 Discussion

Comparison of sources of efficiency: The main source of efficiency of Livia [37] is from reduced NoC traversal, so their performance gain is highlighted for a system with a large number of cores and NoC nodes. However, the actual number of read/write operations issued to the memory system, irrespective of the memory type, remains constant. While it is true that NoC incurs significant energy costs, memory operations are no less expensive, particularly for LLCs and DRAMs, and become dominant in smaller systems. In this respect, MVC can reduce the absolute number of memory requests by creating compact views and making them reusable in caches. MVC can reduce both NoC traversal and total memory accesses, allowing even small-scale systems to benefit from our approach.

Impact of baseline data format: Our experiment assumes a cache line aligned AOS (array of structures) data format. We also evaluate an implementation of dynamic graph BFS that uses a packed data format, where we strip node metadata and provide a dense vector format containing only node key pairs. If the base data format is packed, it intrinsically increases the cache effectiveness, resulting in the CPU outperforming PIM, as shown in Figure 16. It also increases the cache effectiveness of MVC (Frag) because the packed format enables multiple fragments to exist in the same cache line. Interestingly, MVC (View) does not show a significant performance difference between the two formats. This indicates AOS data structures can achieve a performance comparable to those of an idealistic packed structure.

MVC in DBMS: Our experiments use the database and irregular workloads evaluated in the prior works. In this subsection, we discuss the expected performance gain in real-world database management systems. Specifically, we study RocksDB [52], a widely-used DBMS developed by Facebook. RocksDB primarily consists of two data structures, in-DRAM skiplist-based memtables and in-storage LMS-tree-based SSTables. As suggested by Yoo *et al.* [59], memtable costs are dominantly high across varying DB sizes, ranging from 33% to 80%, given a default memtable size of 64MB. Typically, larger memtables are used in server setups, which further increases the relative memtable costs. Our simulation shows reveals a similar speedup for memtable processing, projecting the end-to-end speedup from solely optimizing the memtable to be 1.3× to 2.2×.

Already-dense data structures like B+/LMS trees would not significantly benefit from MVC, unless views are utilized for their auxiliary structures such as a secondary index. However, due to high write and maintenance costs, B+/LMS trees are mainly used for data in persistent memories and storages, where high maintenance costs can amortize the high operation (read/write) cost of storage access. Main memory data structure tends to favor low operation and maintenance costs, employing lightweight structures such as skiplist (RocksDB [52], LevelDB [27], Cassandra [9], Redis [46]),

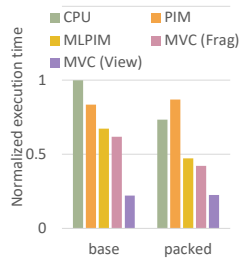


Figure 16: BFS (ubuntu) with different formats.

hash linked-list (Redis [46]), and AVL-tree, at the expense of sparsity. MVC targets such in-memory data structures to mitigate read amplification, a traditionally unavoidable cost for these structures.

Complexity and benefit: While MLPIM and MVC (Frag) require PIM at nearly all layers in the memory hierarchy to collect partial data, we notice that MVC (View) can maintain current performance with PIM only at LLC and DRAM. This is because the dense views can be reused in L1/L2 without PIM and updates usually propagate to LLC for view recreation. Hence, L1/L2 PIM utilization remains low, and omitting them does not cause noticeable performance degradation.

7 RELATED WORK

To the best of our knowledge, this is the first work enabling the coherent reuse of PIM-generated views and demonstrating its wide applicability to various irregular applications. In this section, we discuss some closely related work.

PIM. Over the past few decades, there has been a growing interest in processing in memory (PIM), also known as near-memory computing and in-memory computing. These techniques aim to address the high cost associated with data movement and the challenges related to the memory wall. PIM reduces the data movement by performing computations near the memory, taking advantage of the internal bandwidth of a memory [3, 6, 17, 18, 25, 33, 37, 41–43, 47, 61, 64]. Specialized PIM architectures have been proposed for various acceleration targets, including machine learning [24, 26, 36], sparse data processing [24, 28, 32], and databases [15]. In-memory computing [1, 12, 16, 21, 22, 50] utilizes the physical properties of the memory array to repurpose memories themselves for computation, unlocking massive parallelism and minimizing data movement.

System integration of PIM. Integrating PIM into conventional computers is one of the key challenges in realizing the true value of PIM to the system. Prior work tries to identify and schedule program blocks suitable for PIM execution: PEI [3] uses a locality monitor to evaluate the performance of remote PIM execution, and AMS [54] leverages cache partitioning to determine thread mappings for a heterogeneous memory system. Limited computation performance of PIM has also motivated dynamic work sharing with CPUs [35]. In this work, we focus on data collection tasks, one of the fundamental use cases of PIM working cooperatively with the CPU, while future work can expand the scope to other applications through integration with software memorization conventions. Another effort tries to ensure data freshness for PIM, which can be done through explicit data migration [30], software-based update bookkeeping [55], speculative PIM execution [5], and memory address separation [23]. Our approach differs from the prior efforts in that we support both input and output coherence. Although MVC is based on a multi-layer PIM system, a single-layer PIM system should also benefit from MVC.

Data reorganization using PIM. General and specialized data reorganization has been explored as a potential PIM application. Comparisons with prior data reorganization work are shown in Table 4. Impulse [7] supports application-specific physical address remappings at the memory controller. Access to shadow address space automatically densifies scattered data through bus packaging, but no coherence support is provided between the address spaces,

Table 4: Comparison with prior data reorganization PIM.

	Reorganization scheme	Locality aware	Near memory densification*	Input/output coherence
Impulse [7]	User-defined	-	Y	-/-
Stash [34]	User-defined	(Y)**	-	Y/Y
SAM [58] /				
GS-DRAM [49] /	Fixed-stride	-	Y	-/-
RC-NVM [57]				
Livia [37]	User-defined	Y	-	Y/-
Akin <i>et al.</i> [4]	User-defined	-	Y	-/-
Hong <i>et al.</i> [29]	Linked-List	-	Y	-/-
MVC (this work)	User-defined	Y	Y	Y/Y

* Including LLC/DRAM. ** Limited in the local store near L1.

thereby limiting its applicability to read-only data. Stash [34] enables coherent data mapping to its local store near the private L1 cache. Stash can be thought of as a coalescing approach (Figure 7) at the private caches, and does not reduce data fetches from LLC/memory. We believe filtering redundant data is most effective near the memory that owns it.

Similar to SAM [58], Gather/ScatterDRAM [49] is proposed to accelerate strided accesses at the chip level by augmenting the address bus to drive multiple rows in different chips. RC-NVM [57] also optimizes it by exploiting the symmetry of the crossbar structure of NVM. As discussed in Section 2, these approaches can substantially benefit from reuse opportunities brought by MVC.

Akin *et al.* [4] proposed a data reorganization architecture in HMC, utilizing a mathematical remapping representation of permutations, through manual coherence management using cache flush. Since MVC does not restrict its PE design, it can benefit from such a declarative representation and model to simplify its logic, while providing the reuse opportunities. Hong *et al.* [29] proposed a near-memory accelerator for linked-list traversal on HMC-based memory networks. To maintain coherence with the host-side memory system, they use a combination of write-through (WT) cache and barrier synchronization. MVC enables output reuse, which is particularly beneficial for sophisticated linked-list-based data structures such as skiplists, using host-side caches. It also facilitates coherence support without significantly increasing memory traffic in contrast to WT caches.

8 CONCLUSION

We propose MVC, a new approach that facilitates fully coherent data views in a multi-layer PIM system. We demonstrate that MVC significantly reduces data movement and accelerates irregular workloads, enabling locality-aware reuse of user-defined views. The input and output coherence of PIM operations is guaranteed by participating in existing coherence protocols, assisted by our introduced coherence extensions to expand the sharer list capacity. We define reusable views that facilitate new software optimizations on top of existing PIM approaches, which can be easily extended for other applications. We show MVC improves the performance of irregular applications by 3.27× over CPU and 1.60× over state-of-the-art MLPIM with only 1.8% cache area overhead.

ACKNOWLEDGMENTS

We thank the anonymous reviewers and shepherd for their suggestions which helped improve this paper. This work was supported in part by JSPS KAKENHI Grant Number JP22K21284 and JST PRESTO Grant Number JPMJPR22P7, Japan.

REFERENCES

- [1] S. Aga, S. Jeloka, A. Subramaniyan, S. Narayanasamy, D. Blaauw, and R. Das. 2017. Compute Caches. In *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 481–492. <https://doi.org/10.1109/HPCA.2017.21>
- [2] Anant Agarwal, Richard Simoni, John Hennessy, and Mark Horowitz. 1988. An evaluation of directory schemes for cache coherence. *ACM SIGARCH Computer Architecture News* 16, 2 (1988), 280–298.
- [3] Junwhan Ahn, Sungjoo Yoo, Onur Mutlu, and Kiyoung Choi. 2015. PIM-enabled Instructions: A Low-overhead, Locality-aware Processing-in-memory Architecture. In *Proceedings of the 42Nd Annual International Symposium on Computer Architecture (ISCA '15)*.
- [4] Berkin Akin, Franz Franchetti, and James C Hoe. 2015. Data reorganization in memory using 3D-stacked DRAM. *ACM SIGARCH Computer Architecture News* 43, 35 (2015), 131–143.
- [5] Amirali Boroumand, Saugata Ghose, Minesh Patel, Hasan Hassan, Brandon Lucia, Kevin Hsieh, Krishna T. Malladi, Hongzhong Zheng, and Onur Mutlu. 2017. LazyPIM: An Efficient Cache Coherence Mechanism for Processing-in-Memory. *IEEE Computer Architecture Letters* 16, 1 (2017), 46–50. <https://doi.org/10.1109/LCA.2016.2577557>
- [6] Jay B. Brockman, Shyamkumar Thoziyoor, Shannon K. Kuntz, and Peter M. Kogge. 2004. A Low Cost, Multithreaded Processing-in-memory System. In *Proceedings of the 3rd Workshop on Memory Performance Issues: In Conjunction with the 31st International Symposium on Computer Architecture (Munich, Germany) (WMPPI '04)*. ACM, New York, NY, USA, 16–22. <https://doi.org/10.1145/1054943.1054946>
- [7] John Carter, Wilson Hsieh, Leigh Stoller, Mark Swanson, Lixin Zhang, Erik Brunvand, Al Davis, Chen-Chi Kuo, Ravindra Kuramkote, Michael Parker, et al. 1999. Impulse: Building a smarter memory controller. In *Proceedings Fifth International Symposium on High-Performance Computer Architecture*. IEEE, 70–79.
- [8] Daniel Rodrigues Carvalho and André Seznec. 2021. Understanding cache compression. *ACM Transactions on Architecture and Code Optimization (TACO)* 18, 3 (2021), 1–27.
- [9] Apache Cassandra. 2014. Apache cassandra. Website. Available online at <http://planetcassandra.org/what-is-apache-cassandra> 13 (2014).
- [10] David Chaiken, Craig Fields, Kiyoshi Kurihara, and Anant Agarwal. 1990. Directory-based cache coherence in large-scale multiprocessors. *Computer* 23, 6 (1990), 49–58.
- [11] David Chaiken, John Kubiatowicz, and Anant Agarwal. 1991. LimitLESS directories: A scalable cache coherence scheme. *ACM SIGARCH Computer Architecture News* 19, 2 (1991), 224–234.
- [12] Ping Chi, Shuangchen Li, and Cong Xu. 2016. PRIME : A Novel Processing-in-memory Architecture for Neural Network Computation in ReRAM-based Main Memory. In *IEEE International Symposium on Computer Architecture*. IEEE, 27–39. <https://doi.org/10.1109/ISCA.2016.13>
- [13] Henry Cook, Krste Asanovic, and David A Patterson. 2009. *Virtual local stores: Enabling software-managed memory hierarchies in mainstream computing environments*. Technical Report. Technical Report No. UCB/EECS-2009-131.
- [14] David L Dill, Andreas J Drexler, Alan J Hu, and C Han Yang. 1992. Protocol verification as a hardware design aid. In *ICCD*, Vol. 92. Citeseer, 522–525.
- [15] Mario Drumond, Alexandros Daglis, Nooshin Mirzadeh, Dmitrii Ustiugov, Javier Picorel, Babak Falsafi, Boris Grot, and Dionisios Pnevmatikatos. 2017. The mon-drian data engine. *ACM SIGARCH Computer Architecture News* 45, 2 (2017), 639–651.
- [16] C. Eckert, X. Wang, J. Wang, A. Subramaniyan, R. Iyer, D. Sylvester, D. Blaauw, and R. Das. 2018. Neural Cache: Bit-Serial In-Cache Acceleration of Deep Neural Networks. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. 383–396. <https://doi.org/10.1109/ISCA.2018.00040>
- [17] A. Farmahini-Farahani, Jung Ho Ahn, K. Morrow, and Nam Sung Kim. 2015. NDA: Near-DRAM acceleration architecture leveraging commodity DRAM devices and standard memory modules. In *High Performance Computer Architecture (HPCA)*, 2015 IEEE 21st International Symposium on.
- [18] Basilio B. Fraguera, Jose Renau, Paul Feautrier, David Padua, and Josep Torrellas. 2003. Programming the FlexRAM Parallel Intelligent Memory System. *SIGPLAN Not.* 38, 10 (June 2003), 49–60. <https://doi.org/10.1145/966049.781505>
- [19] Daichi Fujiki, Niladrish Chatterjee, Donghyuk Lee, and Mike O'Connor. 2019. Near-memory data transformation for efficient sparse matrix multi-vector multiplication. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–17.
- [20] Daichi Fujiki, Alireza Khadem, Scott Mahlke, and Reetuparna Das. 2022. Multi-Layer In-Memory Processing. In *2022 55th IEEE/ACM International Symposium*

- on *Microarchitecture (MICRO)*. 920–936. <https://doi.org/10.1109/MICRO56248.2022.00068>
- [21] Daichi Fujiki, Scott Mahlke, and Reetuparna Das. 2018. In-Memory Data Parallel Processor. In *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems* (Williamsburg, VA, USA) (ASPLOS '18). ACM, New York, NY, USA, 1–14. <https://doi.org/10.1145/3173162.3173171>
 - [22] Daichi Fujiki, Scott Mahlke, and Reetuparna Das. 2019. Duality cache for data parallel acceleration. In *Proceedings of the 46th International Symposium on Computer Architecture*. 397–410.
 - [23] Daichi Fujiki, Xiaowei Wang, Arun Subramaniyan, and Reetuparna Das. 2021. In-/near-memory Computing. *Synthesis Lectures on Computer Architecture* 16, 2 (2021), 1–140.
 - [24] Mingyu Gao, Grant Ayers, and Christos Kozyrakis. 2015. Practical near-data processing for in-memory analytics frameworks. In *2015 International Conference on Parallel Architecture and Compilation (PACT)*. IEEE, 113–124.
 - [25] Mingyu Gao and Christos Kozyrakis. 2016. HRL: Efficient and flexible reconfigurable logic for near-data processing. In *2016 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. Ieee, 126–137.
 - [26] Mingyu Gao, Jing Pu, Xuan Yang, Mark Horowitz, and Christos Kozyrakis. 2017. Tetris: Scalable and efficient neural network acceleration with 3d memory. In *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems*. 751–764.
 - [27] Sanjay Ghemawat and Jeff Dean. 2011. LevelDB. <https://github.com/google/leveldb>
 - [28] Christina Giannoula, Ivan Fernandez, Juan Gómez Luna, Nectarios Koziris, Georgios Goumas, and Onur Mutlu. 2022. SparseP: Towards efficient sparse matrix vector multiplication on real processing-in-memory architectures. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 6, 1 (2022), 1–49.
 - [29] Byungchul Hong, Gwangsun Kim, Jung Ho Ahn, Yongkee Kwon, Hongsik Kim, and John Kim. 2016. Accelerating linked-list traversal through near-data processing. In *Proceedings of the 2016 International Conference on Parallel Architectures and Compilation*. 113–124.
 - [30] Kevin Hsieh, Eiman Ebrahim, Gwangsun Kim, Niladrish Chatterjee, Mike O'Connor, Nandita Vijaykumar, Onur Mutlu, and Stephen W. Keckler. 2016. Transparent Offloading and Mapping (TOM): Enabling Programmer-Transparent Near-Data Processing in GPU Systems. In *Proceedings - 2016 43rd International Symposium on Computer Architecture, ISCA 2016*. <https://doi.org/10.1109/ISCA.2016.27>
 - [31] WiredTiger Inc. 2012. WiredTiger Storage Engine. <https://www.mongodb.com/docs/manual/core/wiredtiger/>.
 - [32] Liu Ke, Udit Gupta, Benjamin Youngjae Cho, David Brooks, Vikas Chandra, Utku Diril, Amin Firoozshahian, Kim Haezwood, Bill Jia, Hsien-Hsin S Lee, et al. 2020. Recnmp: Accelerating personalized recommendation with near-memory processing. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 790–803.
 - [33] Duckhwan Kim, Jaeha Kung, Sek Chai, Sudhakar Yalamanchili, and Saibal Mukhopadhyay. 2016. Neurocube: A Programmable Digital Neuromorphic Architecture with High-Density 3D Memory. In *Proceedings of ISCA*, Vol. 43.
 - [34] Rakesh Komuravelli, Matthew D. Sinclair, Johnathan Alsop, Muhammad Huzaifa, Maria Kotsifakou, Prakash Srivastava, Sarita V. Adve, and Vikram S. Adve. 2015. Stash: Have Your Scratchpad and Cache It Too. In *Proceedings of the 42nd Annual International Symposium on Computer Architecture (Portland, Oregon) (ISCA '15)*. Association for Computing Machinery, New York, NY, USA, 707–719. <https://doi.org/10.1145/2749469.2750374>
 - [35] Gunjae Koo, Kiran Kumar Matam, Te I, HV Krishna Giri Narra, Jing Li, Hung-Wei Tseng, Steven Swanson, and Murali Annavaram. 2017. Summarizer: trading communication with computing near storage. In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture*. 219–231.
 - [36] Youngeun Kwon, Yunjae Lee, and Minsoo Rhu. 2019. Tensordimm: A practical near-memory processing architecture for embeddings and tensor operations in deep learning. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. 740–753.
 - [37] Elliot Lockerman, Axel Feldmann, Mohammad Bakhshalipour, Alexandru Stanescu, Shashwat Gupta, Daniel Sanchez, and Nathan Beckmann. 2020. Livia: Data-Centric Computing Throughout the Memory Hierarchy. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (Lausanne, Switzerland) (ASPLOS '20)*. Association for Computing Machinery, New York, NY, USA, 417–433. <https://doi.org/10.1145/3373376.3378497>
 - [38] Andrew McCrabb, Hellina Nigatu, Absalat Getachew, and Valeria Bertacco. 2022. DyGraph: A Dynamic Graph Generator and Benchmark Suite. In *Proceedings of the 5th ACM SIGMOD Joint International Workshop on Graph Data Management Experiences & Systems (GRADES) and Network Data Analytics (NDA)* (Philadelphia, Pennsylvania) (GRADES-NDA '22). Association for Computing Machinery, New York, NY, USA, Article 7, 8 pages. <https://doi.org/10.1145/3534540.3534692>
 - [39] Igor Melatti, Robert Palmer, Geoffrey Sawaya, Yu Yang, Robert M Kirby, and Ganesh Gopalakrishnan. 2009. Parallel and distributed model checking in eddy. *International Journal on Software Tools for Technology Transfer* 11 (2009), 13–25.
 - [40] Naveen Muralimanohar, Rajeev Balasubramanian, and Norman P Jouppi. 2009. CACTI 6.0: A tool to model large caches. *HP laboratories* (2009), 22–31.
 - [41] Mark Oskin, Frederic T Chong, Timothy Sherwood, Mark Oskin, Frederic T Chong, and Timothy Sherwood. 1998. Active Pages: A Computation Model for Intelligent Memory. *ACM SIGARCH Computer Architecture News* 26, 3 (1998), 192–203. <https://doi.org/10.1145/279358.279387>
 - [42] D. Patterson, T. Anderson, N. Cardwell, R. Fromm, K. Keeton, C. Kozyrakis, R. Thomas, and K. Yelick. 1997. A case for intelligent RAM. *Micro, IEEE* (1997).
 - [43] S.H. Pugsley, J. Jests, Huilui Zhang, R. Balasubramanian, V. Srinivasan, A. Buyuktosunoglu, A. Davis, and Feifei Li. 2014. NDC: Analyzing the impact of 3D-stacked memory+logic devices on MapReduce workloads. In *Performance Analysis of Systems and Software (ISPASS), 2014 IEEE International Symposium on*.
 - [44] Daniel Sanchez and Christos Kozyrakis. 2010. The ZCache: Decoupling ways and associativity. In *2010 43rd Annual IEEE/ACM International Symposium on Microarchitecture*. IEEE, 187–198.
 - [45] Daniel Sanchez and Christos Kozyrakis. 2013. ZSim: Fast and accurate microarchitectural simulation of thousand-core systems. *ACM SIGARCH Computer architecture news* 41, 3 (2013), 475–486.
 - [46] S Sanfilippo. 2009. Redis In-memory Data Structure Server. <https://redis.io/>.
 - [47] Vivek Seshadri, Yoongu Kim, Chris Fallin, Donghyuk Lee, Rachata Ausavarungnirun, Gennady Pekhimenko, Yixin Luo, Onur Mutlu, Phillip B. Gibbons, Michael A. Kozuch, and Todd C. Mowry. [n. d.]. RowClone: Fast and Energy-efficient in-DRAM Bulk Data Copy and Initialization. In *Proceedings of the 46th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-46)*.
 - [48] Vivek Seshadri, Donghyuk Lee, Thomas Mullins, Hasan Hassan, Amirali Boroumand, Jeremie Kim, Michael A Kozuch, Onur Mutlu, Phillip B Gibbons, and Todd C Mowry. 2017. Ambit: In-memory accelerator for bulk bitwise operations using commodity DRAM technology. In *2017 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 273–287.
 - [49] Vivek Seshadri, Thomas Mullins, Amirali Boroumand, Onur Mutlu, Phillip B Gibbons, Michael A. Kozuch, and Todd C Mowry. 2015. Gather-Scatter DRAM: In-DRAM address translation to improve the spatial locality of non-unit strided accesses. In *2015 48th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 267–280. <https://doi.org/10.1145/2830772.2830820>
 - [50] Ali Shafiee, Anirban Nag, Naveen Muralimanohar, and Rajeev Balasubramanian. 2016. ISAAC: A Convolutional Neural Network Accelerator with In-Situ Analog Arithmetic in Crossbars. *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)* (jun 2016), 14–26. <https://doi.org/10.1109/ISCA.2016.12>
 - [51] Linghao Song, Xuehai Qian, Hai Li, and Yiran Chen. 2017. PipeLayer: A Pipelined ReRAM-Based Accelerator for Deep Learning. In *Proceedings - International Symposium on High-Performance Computer Architecture*. 541–552. <https://doi.org/10.1109/HPCA.2017.55>
 - [52] Facebook RocksDB team. 2012. A Persistent Key-Value Store for Fast Storage Environments. <https://rocksdb.org/>
 - [53] Po-An Tsai, Nathan Beckmann, and Daniel Sanchez. 2017. Jenga: Software-defined cache hierarchies. In *Proceedings of the 44th Annual International Symposium on Computer Architecture*. 652–665.
 - [54] Po-An Tsai, Changping Chen, and Daniel Sanchez. 2018. Adaptive Scheduling for Systems with Asymmetric Memory Hierarchies. In *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 641–654. <https://doi.org/10.1109/MICRO.2018.00058>
 - [55] Tobias Vinçon, Christian Knödler, Leonardo Solis-Vasquez, Arthur Bernhardt, Sajjad Tamimi, Lukas Weber, Florian Stock, Andreas Koch, and Ilia Petrov. 2022. Near-data processing in database systems on native computational storage under htp workloads. *Proceedings of the VLDB Endowment* 15, 10 (2022), 1991–2004.
 - [56] Minjie Wang, Da Zheng, Zihao Ye, Quan Gan, Mufei Li, Xiang Song, Jinjing Zhou, Chao Ma, Lingfan Yu, Yu Gai, et al. 2019. Deep graph library: A graph-centric, highly-performant package for graph neural networks. *arXiv preprint arXiv:1909.01315* (2019).
 - [57] Peng Wang, Shuo Li, Guangyu Sun, Xiaoyang Wang, Yiran Chen, Hai Li, Jason Cong, Nong Xiao, and Tao Zhang. 2018. RC-NVM: Enabling Symmetric Row and Column Memory Accesses for In-memory Databases. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 518–530. <https://doi.org/10.1109/HPCA.2018.00051>
 - [58] Xin Xin, Yanan Guo, Youtao Zhang, and Jun Yang. 2021. SAM: accelerating strided memory accesses. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. 324–336.
 - [59] Seehwan Yoo, Hojin Shin, Sunghyun Lee, and Jongmoo Choi. 2022. A Read Performance Analysis with Storage Hierarchy in Modern KVS: A RocksDB Case. In *2022 IEEE 11th Non-Volatile Memory Systems and Applications Symposium (NVMSA)*. IEEE, 45–50.
 - [60] Dongping Zhang, Nuwan Jayasena, Alexander Lyashevsky, Joseph I. Greathouse, Lifan Xu, and Michael Ignatowski. 2014. In *Proceedings of the 23rd international symposium on High-performance parallel and distributed computing*. 85–98.

- [61] Dongping Zhang, Nuwan Jayasena, Alexander Lyashevsky, Joseph L. Greathouse, Lifan Xu, and Michael Ignatowski. 2014. TOP-PIM: Throughput-oriented Programmable Processing in Memory. In *Proceedings of the 23rd International Symposium on High-performance Parallel and Distributed Computing (HPDC '14)*.
- [62] Muhan Zhang and Yixin Chen. 2018. Link prediction based on graph neural networks. *Advances in Neural Information Processing Systems* 31 (2018), 5165–5175.
- [63] Muhan Zhang, Pan Li, Yinglong Xia, Kai Wang, and Long Jin. 2020. Revisiting graph neural networks for link prediction. *arXiv preprint arXiv:2010.16103* (2020).
- [64] Qiuling Zhu, B. Akin, H.E. Sumbul, F. Sadi, J.C. Hoe, L. Pileggi, and F. Franchetti. 2013. A 3D-stacked logic-in-memory accelerator for application-specific data intensive computing. In *3D Systems Integration Conference (3DIC), 2013 IEEE International*.